

PROJEKTBEKRIVELSE, SEP3

Leveringsplanlægger

Leveringskoordinering for en forhandler med flere lagre

UDDANNELSE	VIA University College, Software Engineering, 3. semester
STUDERENDE	Abdirahman Isse Ibrahim Mahamed, studienummer 364963, TS-SEP3-A26
VEJLEDERE	Jakob Trigger Knop og Surayya Urazimbetova
VIRKSOMHED	ZNG, kontaktperson Abdiqani Mohamud
DATO	23. september 2026
KILDEKODE	https://github.com/iabdirahman120/sep3-leveringsplanlaegger

1. Baggrund

En dansk forhandler af hvidevarer leverer fra flere lagre med egne chauffører. Koordineringen sker i dag i delte To Do-lister med én afkrydsning pr. levering.

Det giver tre problemer, som alle kan mærkes i driften:

Ingen historik. Når en levering mislykkes, fordi ingen er hjemme, eller fordi adgangen nægtes, oprettes opgaven bare igen. Forsøget forsvinder, og ingen kan bagefter se, at kunden nu er kørt forgæves til to gange.

Intet ejerskab. Der findes ikke noget svar på, hvem der flyttede en ordre og hvornår, eller hvem der kvitterede for varen. Ved en tvist står påstand mod påstand.

Intet overblik. Disponenten kan ikke se belastningen pr. chauffør eller pr. lager og opdager først en forsinkelse, når kunden ringer.

Domænet er hentet fra et reelt behov, jeg har arbejdet med gennem ZNG. Systemet bygges fra bunden i Java og C#. Jeg genbruger problemforståelsen, datamodellen og forretningsreglerne, men ingen kode. Projektet laves af én person.

2. Problemformulering

Hvordan kan der udvikles et distribueret system i Java og C#, hvor reglerne for en leveringsordres livscyklus håndhæves ét sted og ikke kan omgås, også når flere brugere handler på den samme ordre samtidig fra hver sin enhed?

Underspørgsmål

1. Hvor i et system med tre processer skal reglerne for, hvem der må flytte en ordre, placeres, for at afgørelsen stadig er gyldig i det øjeblik, ændringen skrives?
2. Hvilken kommunikationsform passer til hvilket led mellem browser, app-server og domæneserver, og hvad koster valget i kompleksitet?
3. Hvordan håndteres to samtidige ændringer på den samme ordre, så præcis én lykkes, og historikken viser præcis én hændelse?
4. Hvordan kan en medarbejders adgang lukkes øjeblikkeligt, og hvordan sikres det, at en chauffør kun kan se egne leveringer, også hvis brugerfladen omgås?

3. Afgrænsning

Med i projektet

- Oprettelse af ordrer med lagertjek og tildeling af chauffør
- Statusforløb med regler for, hvem der må flytte hvad
- Chaufførvisning til telefon med registrering af modtager

- Uredigerbar tidslinje pr. ordre
- Dashboard for disponenten
- Brugerstyring med to roller og øjeblikkelig deaktivering

Ikke med i projektet

- **Kundenotifikationer** via SMS eller e-mail, og offentlig sporingsside
- **Ruteoptimering og kort.** Ordren har adresse og leveringsvindue, men rækkefølgen på chaufførens dag lægges manuelt
- **Integration til ERP eller webshop.** Ordre oprettes i systemet
- **Returlogistik.** En mislykket levering registreres, men varens vej tilbage til lageret følges ikke
- **Udrulning hos rigtige brugere.** Der testes med simulerede lagre og chauffører
- Nulstilling af adgangskode via e-mail, mørkt tema og lignende bekvemmeligheder

4. Aktører og brugere

Aktør	Rolle	Behov
Disponent	Primær bruger. Kontorpersonale, der planlægger og overvåger	Dashboard til desktop med overblik på tværs af lagre
Chauffør	Sekundær bruger. Ansat, knyttet til ét lager	Telefonvenlig visning med få tryk, kun egne leveringer
Kunde	Modtager varen, men er ikke bruger af systemet	Kundetypen modelleres, fordi den styrer leveringsvindue og kontaktfelter

En privat husstand kræver et tidsvindue og en person hjemme, en byggeplads tager imod ved kantsten i arbejdstiden, og en virksomhed har en varemodtagelse. Derfor er kundetypen ikke kosmetisk.

5. Krav

5.1 Funktionelle krav

Skrevet som user stories og prioriteret efter MoSCoW.

#	User story	Prioritet
F1	Som bruger vil jeg logge ind og møde den visning, min rolle giver, så jeg kun ser det, jeg skal bruge	Skal
F2	Som disponent vil jeg oprette en ordre med kunde, lager, varelinjer og leveringsvindue, så leveringen kan planlægges	Skal
F3	Som disponent vil jeg forhindres i at oprette en ordre, der overstiger lagerbeholdningen, så vi ikke lover varer, vi ikke har	Skal
F4	Som disponent vil jeg kun kunne vælge chauffører fra ordrens eget lager, så ordren ikke havner hos en, der ikke kan hente den	Skal
F5	Som chauffør vil jeg se mine egne leveringer på telefonen, så jeg kan arbejde uden en computer	Skal
F6	Som chauffør vil jeg flytte ordren gennem statusforløbet, så disponenten kan følge med	Skal
F7	Som chauffør vil jeg registrere modtagerens navn ved levering, så der er bevis for, hvem der fik varen	Skal
F8	Som disponent vil jeg se en tidslinje pr. ordre med aktør og tidsstempel, som ikke kan redigeres, så ansvaret kan placeres bagefter	Skal
F9	Som disponent vil jeg se utildelte og forsinkede ordrer samt belastning pr. chauffør, så jeg kan gribe ind før kunden ringer	Skal
F10	Som disponent vil jeg deaktivere en chauffør, så adgangen lukkes med det samme og de tildelte ordrer frigives	Skal
F11	Som disponent vil jeg planlægge en mislykket levering igen, uden at ordrens historik går tabt, så forgæves forsøg kan tælles	Bør
F12	Som disponent vil jeg se dashboardet opdatere sig selv, når en chauffør flytter en ordre, så jeg ikke skal genindlæse	Bør
F13	Som disponent vil jeg filtrere ordrelisten på status og lager, så jeg hurtigt finder de relevante	Bør
F14	Som disponent vil jeg modtage ændringer som en løbende strøm fra domæneserveren, så dashboardet er opdateret uden at spørge efter det	Kunne
F15	Som chauffør vil jeg kunne arbejde uden netforbindelse og sende, når jeg er online igen	Får ikke

F15 er med for at markere grænsen. Offline-understøttelse kræver en kø på enheden og konfliktløsning ved synkronisering, og det hører ikke til i et semesterprojekt på denne størrelse.

5.2 Ikke-funktionelle krav

#	Krav
I1	En statusændring skal være synlig på et åbent dashboard senest to sekunder efter, den er skrevet
I2	Systemet skal kunne håndtere 20 chauffører og 200 aktive ordrer pr. lager, uden at ordrelisten svarer langsommere end ét sekund
I3	Ændrer to brugere den samme ordre samtidig, skal præcis én skrivning lykkes, og tidslinjen skal vise præcis én hændelse
I4	En chauffør må hverken kunne se eller ændre en anden chaufførs ordrer, heller ikke ved at kalde REST-laget uden om brugerfladen
I5	En deaktiveret bruger skal være afvist ved første kald efter deaktivering
I6	Domæneserverens port må ikke være tilgængelig uden for det interne netværk, og kommunikationen skal kunne køre krypteret
I7	Chaufførvisningen skal kunne betjenes med én hånd, og en statusændring må højst kræve to tryk
I8	Systemet skal kunne startes på en ny maskine ud fra dokumentationen og docker compose alene

I3 og I4 er de to, der testes eksplicit, og I3 vises i demovideoen.

6. Foreløbig arkitektur

Tre processer og én database. Alle skrivinger går gennem samme vej.

Klient	Blazor WebAssembly (C#), admin-shell og driver-shell
	REST + SignalR
App-server	ASP.NET Core (C#), cookie og HMAC, rate limit, KPI'er
	gRPC
Domæneserver	Spring Boot (Java), statusmaskine, ejerskab, lagerregler, audit
	JPA
Database	PostgreSQL i Docker

Del	Teknologi	Ansvar
Klient	C#, Blazor WebAssembly	Dashboard til desktop og chaufførvisning til telefon, to shells i én app
App-server	C#, ASP.NET Core	Session-cookie med HMAC, rate limit på login, REST-kontrakt, SignalR-push, validering af input, aggregering af dashboard-tal
Domæneserver	Java, Spring Boot, gRPC, JPA	Ejer databasen inklusive brugere og sessioner. Statusmaskine, ejerskabstjek, lagermatch, audit-log, transaktioner og låsning
Database	PostgreSQL	Ti tabeller. CHECK-constraints på lager og leveringsbevis, versionskolonne på ordrer

Netværksteknologier: REST og SignalR mellem browser og C#-serveren, gRPC mellem C#-serveren og Java-serveren. Kravet om mindst to opfyldes af REST og gRPC alene, og SignalR er et tillæg.

6.1 Statusforløbet

Syv tilstande med én overgangstabel, der findes ét sted og håndhæves på serveren. Brugerfladen viser kun de knapper, brugeren må trykke på, men serveren tjekker igen, før der skrives.

```
pending -> assigned -> picked_up -> in_transit -> delivered
                        |               |
                        +----> failed <+
                        |
                        +----> assigned (planlægges igen)
```

pending, assigned og failed kan desuden gå til cancelled.
delivered og cancelled er slutstationer.

failed er med vilje ikke en sluttilstand. En mislykket levering er normal i hvidevarelogistik og skal kunne planlægges igen, uden at historikken går tabt.

6.2 Tre beslutninger, jeg vil forsvare

Statusmaskinen ligger i Java, i samme transaktion som skrivningen. Afgørelsen "må denne bruger flytte denne ordre" gælder kun det øjebliksbillede, der skrives mod. C# henter de tilladte overgange over gRPC og tegner knapper ud fra svaret. Overgangstabellen dubleres ikke, og Java-API'et kan ikke misbruges udenom.

Optimistisk låsning hele vejen. Klienten tegnede knapper på en bestemt version af ordren. Versionen følger med tilbage gennem alle tre processer. Er ordren ændret undervejs, afvises skrivningen, og brugeren bliver bedt om at genindlæse. Databasens CHECK-constraints er bagstopper mod oversalg.

Sessioner er en tabel, ikke et JWT. Når en disponent deaktiverer en chauffør, skal chaufføren være afvist ved næste kald. Det kan et signeret token uden databaseopslag ikke levere. Prisen er ét opslag pr. kald, og den er bevidst betalt.

6.3 Teknologivalg holdt op mod alternativer

Valg	Alternativ	Begrundelse
gRPC mellem C# og Java	REST med JSON, RabbitMQ, rå sockets	Kontrakten skrives først i <code>.proto</code> , og koden genereres til begge sprog, så de to sider ikke kan blive uenige om formatet. En beskedkø tilføjer drift uden at løse et behov, fordi alle skrivninger går gennem én C#-proces
REST og SignalR mellem browser og C#	GraphQL, rå WebSocket	Dashboardets forespørgsler er faste, så GraphQL løser et problem, jeg ikke har. SignalR giver push uden egen protokol
Blazor WebAssembly hostet af API'et	Blazor Server, .NET MAUI	REST-hoppet bliver ægte og synligt i browseren. Blazor Server kræver en konstant WebSocket, hvilket er skrøbeligt i en varevogn. MAUI kræver Mac, provisioning og en tredje pipeline
PostgreSQL	SQLite, MongoDB	Ægte interaktive transaktioner og moden JPA-dialekt. Data er relaterede: kunde, lager, ordre, varelinje, hændelse
Optimistisk låsning	Pessimistisk lås med <code>SELECT FOR UPDATE</code>	En pessimistisk lås beskytter kun inde i Java. Den fanger ikke, at klienten handlede på et forældet billede, hentet to netværkshop væk
Regler i Java	Regler i C#, eller dubleret begge steder	Én kilde til sandhed. Dubleres tabellen, driver de to fra hinanden

Jeg har gennem ZNG dokumentation for et kørende system i samme domæne, bygget serverløst på Cloudflare. Dets kendte begrænsninger, blandt andet manglende interaktive transaktioner, bruges i rapporten som alternativer med målte konsekvenser i stedet for hypoteser.

7. Metode

Scrum med sprints på to uger. Product backlog, sprint backlog og burndown føres som GitHub Issues og GitHub Projects i det repo, `github.txt` peger på, så proces og kode ligger samme sted.

Projektet laves af én person, og det skal Scrum tilpasses uden at blive tom ceremoni. Rollerne product owner, scrum master og udvikler falder sammen i mig, og der er derfor ingen daglige møder. Det følgende holdes:

Element	Sådan
Product backlog	Hvert krav i afsnit 5 bliver et eller flere issues, prioriteret efter MoSCoW
Sprint planning	Første mandag i sprinten. Sprint backlog vælges og fryses
Sprint backlog	Eget board i GitHub Projects. Højst to issues i gang ad gangen
Daily	Erstattet af en skriftlig log mandag, onsdag og fredag: hvad blev færdigt, hvad blokerer
Sprint review	Ved sprintens slutning. Det færdige vises, og vejledermøder bruges som review med interessant
Retrospektiv	Skrives ned efter hver sprint i tre linjer: behold, stop, prøv
Burndown	Hentes fra GitHub Projects og gemmes som skærbillede ved hver sprintafslutning

Afvigelserne fra ren Scrum, altså de sammenfaldende roller og det manglende daglige møde, skrives ind i rapportens metodeafsnit med begrundelse i stedet for at blive udeladt.

Kontrakt først. `.proto`-filen og REST-kontrakten skrives, før der implementeres. **Gående skelet** senest i sprint 2: login og ordreliste virker end-to-end gennem alle tre processer. Derefter lodrette skiver, altså én funktion gennem alle lag ad gangen.

Definition of Done: enhedstest af hele overgangstabellen uden database, integrationstest af gRPC-kontrakten, og e2e-test af de tre vigtigste flows.

8. Tidsplan

Sprint	Slutter	Mål
1	8/10	Arkitektur afleveret: C1, C2, C3 og datamodel. <code>.proto</code> og REST-kontrakt færdige. docker-compose kører
2	22/10	Gående skelet: login og ordreliste virker end-to-end gennem alle tre processer
3	5/11	Opret ordre med lagertjek, tildel chauffør, statusmaskine og audit-log
4	19/11	Chaufførvisning på telefon, leveringsbevis, dashboard med roller, deaktivering
5	3/12	Test, samtidighedstest optaget, dokumentation
6	Efter aftale	Rapport, kildekode, GitHub-link og videodemonstration

Sprint 1 er lagt, så den slutter på arkitekturfristen 8. oktober. Planen bygger på cirka otte timer om ugen ved siden af de øvrige fag.

9. Risici

Sandsynlighed og konsekvens vurderet fra 1 til 5.

#	Risiko	S	K	Tal	Håndtering
R1	Tre kodebaser, en .proto-fil og Docker integreres først til sidst	4	4	16	Gående skelet i sprint 2, derefter lodrette skiver. docker-compose fra dag ét
R2	For stort omfang for én person	4	4	16	Kun kravene mærket Skal indgår i første version. Resten er Bør og Kunne og ryger først
R3	Tid, fordi fire andre fag kører samtidig	4	4	16	Sprintgrænser lagt uden om afleveringer i de andre fag
R4	Reglerne dubleres i C# og Java og driver fra hinanden	3	4	12	Overgangstabellen findes kun i Java. C# henter de tilladte overgange over gRPC
R5	gRPC og Blazor er nye for mig	3	3	9	gRPC indgår i DSY1 i samme periode. Blazor øves i sprint 1 på det gående skelet
R6	Domænet ender som simpel CRUD	3	4	12	Statusmaskine, ejerskab, optimistisk låsning og audit er kernen, ikke formularerne
R7	Scope vokser med notifikationer, kort eller beskedkø	3	3	9	Afsnit 3 er kontrakten. Strækmål først, når alt mærket Skal er i videoen

10. Omverden og bæredygtighed

Systemet påvirker fire grupper.

Disponenterne får en hverdag med færre hastesager. Når en forsinkelse ses på dashboardet i stedet for at komme ind som et opkald, bliver arbejdet planlagt i stedet for akut.

Chaufførerne får dokumentation for, hvad de har gjort. I dag står påstand mod påstand, hvis en kunde siger, at varen aldrig kom. Tidslinjen og modtagerens navn beskytter dem lige så meget som forhandleren.

Kunderne oplever færre forgæves leveringer, fordi et mislykket forsøg ikke længere forsvinder, men tælles og planlægges igen.

Miljømæssigt er effekten indirekte, men reel. En forgæves levering er en kørsel helt uden værdi, og hvidevarer køres i varevogn. Når forgæves forsøg bliver synlige og kan tælles, kan årsagerne bag dem rettes, og det er kørte kilometer, der spares.

Om data. Systemet logger, hvem der flyttede hvilken ordre og hvornår, og det er personoplysninger om ansatte. Loggen er bevidst afgrænset til handlinger i systemet. Der registreres ikke position, køretid eller andet om, hvor chaufføren befinder sig. Formålet er at kunne placere ansvaret for en levering, ikke at måle den enkeltes tempo, og den grænse skrives ind i rapporten.

11. Dækning af kravene til SEP3

Krav	Hvordan
1 Metode	Scrum, backlog og sprints i GitHub Projects
2 Domæne	Dette dokument
3 Versionskontrol	Git fra første commit, se link på forsiden
4 Distribueret	Tre processer, som kan køre på hver sin maskine
5 Java og C#	Java: domæneserver. C#: klient og app-server
6 Server til server	C#-serveren kalder Java-serveren over gRPC
7 To netværksteknologier	REST og gRPC, plus SignalR
8 GUI	Blazor med to visninger
9 Database	PostgreSQL
10 Teknologidiskussion	Afsnit 6.3
11 til 15	Afsnit 12

12. Leverancer

Leverance	Indhold
Rapport	8 til 12 sider ekskl. bilag, hvor én side er 2400 tegn. Arkitektur på højst tre sider
Kildekode	Hele repoet som .zip
<code>github.txt</code>	Link til repoet, som også rummer Scrum-artefakterne
<code>demo.txt</code>	Link til video på højst tre minutter

Videoen viser: login i begge roller, opret og tildel ordre, chauffør leverer fra telefonen, tidslinje, deaktivering, og til sidst samtidighedstesten fra I3, hvor to browsere trykker på den samme ordre, den ene afvises, tidslinjen viser præcis én hændelse, og logs fra begge servere står side om side.

13. Referencer

- Coulouris, G., Dollimore, J., Kindberg, T. og Blair, G. *Distributed Systems: Concepts and Design*, 5. udgave. Addison-Wesley.
- Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley. Mønstret Optimistic Offline Lock ligger til grund for afsnit 6.2.

- Newman, S. *Building Microservices*, 2. udgave. O'Reilly. Brugt til afgrænsningen af ansvar mellem app-server og domæneserver.
- gRPC Documentation. <https://grpc.io/docs/>
- Protocol Buffers Language Guide, proto3. <https://protobuf.dev/programming-guides/proto3/>
- Spring Boot Reference Documentation og Spring Data JPA. <https://docs.spring.io/>
- Microsoft Learn: ASP.NET Core, Blazor og SignalR. <https://learn.microsoft.com/aspnet/core/>
- PostgreSQL Documentation. <https://www.postgresql.org/docs/>
- SEP3 Requirements og introduktions-slides. itslearning, TS-SEP3-A26.

14. Spørgsmål til vejlederne

1. Godkendes domænet som beskrevet i afsnit 1 til 5?
2. Tæller SignalR som en selvstændig netværksteknologi i krav 7? Kravet er opfyldt af REST og gRPC uanset svaret.
3. Er Blazor WebAssembly hostet af C#-API'et acceptabelt som klient i krav 4, når den kører i browseren som separat proces?
4. Er én PostgreSQL tilstrækkelig til krav 9?